
CNN-Adaptive Sliding Mode Control and Reinforcement Learning for Autonomous Differential-Drive Robots

Under Environmental Uncertainty — A Digital Twin Approach

AI Clinic Research Project Report

PGE5 — Programme Grande École

Participating Students: Likhita Yerra
Mohamed Oussama Bouriga
Ahmed Ben Aissa
Abdellahi El Moustapha
Thibault Goutorbe

Supervisor: Prof. Vishvjit Thakar

Date of Submission: 30 August 2026

Abstract

Fixed-gain Sliding Mode Control (SMC) cannot simultaneously optimise chattering suppression and disturbance rejection across diverse operating conditions. We propose a CNN-adaptive SMC framework in which a lightweight convolutional network classifies the pre-mission environment from a 64×64 occupancy map and selects a hand-tuned SMC parameter preset at run time. Five uncertainty scenarios are evaluated on a simulated differential-drive robot against classical SMC and three adaptive baselines: fuzzy gain scheduling, oracle map classification, and reinforcement-learning preset selection. CNN-adaptive SMC reduces angular-velocity chattering by 35.7% under sensor noise and improves final tracking-error recovery by up to 32.7% under wheel slip and 14.4% under impulsive disturbance, approaching oracle performance (17.9 mm vs. 15.6 mm). On realistic occupancy maps, EnvironmentCNN reaches 95.1% classification accuracy. The complete pipeline is integrated into a real-time 3D digital twin for live demonstration, controller comparison, and replay.

Keywords: sliding mode control, adaptive control, convolutional neural network, reinforcement learning, gain scheduling, mobile robot, chattering reduction, digital twin.

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Literature Review	2
2	Methodology	3
2.1	Differential-Drive Robot Model	3
2.2	Classical Sliding Mode Controller	3
2.3	CNN Environment Classifier	4
2.4	Adaptive Parameter Selection	4
2.5	Reinforcement Learning Baseline (PPO)	5
2.6	Stability Analysis	5
2.7	Evaluation Protocol	5
3	Results	6
3.1	CNN Classification	6
3.2	Trajectory and Tracking Performance	6
3.3	Quantitative Comparison	6
3.4	Discussion	7
3.5	Digital Twin Demonstration	8
4	Conclusions	12

1 Introduction

1.1 Problem Statement

Autonomous mobile robots navigating in unstructured environments must maintain accurate trajectory tracking despite sensor noise, mechanical disturbances, and surface-induced wheel slip. Sliding Mode Control (SMC) is widely adopted in this domain because of its intrinsic robustness to matched disturbances and parameter variations [1, 2]. However, classical SMC uses a single fixed set of gains that cannot simultaneously optimise chattering suppression, tracking accuracy, and disturbance rejection across all operating regimes: noisy sensors call for smoother, lower-bandwidth actuation, whereas impulsive pushes and wheel slip call for aggressive corrective gains.

This project addresses the research question: *can a learning-based perception and control pipeline improve the adaptability of sliding mode control for a differential-drive mobile robot under heterogeneous uncertainty conditions?* Our contributions are:

- A complete, reproducible CNN-adaptive SMC pipeline from synthetic map generation and CNN training through simulation and quantitative evaluation.
- An *EnvironmentCNN* that classifies five scenario types from occupancy maps, with 100% test accuracy on engineered simple maps and 95.1% on realistic maps, supported by baseline classifier comparisons.
- A multi-controller benchmark against fuzzy-SMC, oracle selection, and RL-adaptive SMC under identical simulation conditions, showing CNN supervision near the oracle ceiling.
- Quantitative analysis of the tracking–smoothness trade-off: scenario-aware presets reduce chattering by up to 35.7% while preserving interpretable, certifiable gain schedules.
- A real-time 3D *digital twin* (React + Three.js frontend, FastAPI WebSocket backend) providing live visualisation, controller tours, side-by-side comparison, replay, and metric export for demonstration and defense.

1.2 Literature Review

Sliding Mode Control for Mobile Robots. SMC provides finite-time convergence and robustness to matched uncertainties [1]. Anti-chattering techniques, including boundary layers, saturation switching, and higher-order SMC, are well established [3, 4]. Adaptive SMC with Lyapunov-based gain tuning can converge slowly or require tight disturbance bounds [2]. Differential-drive platforms in particular benefit from SMC due to their nonholonomic constraints and sensitivity to lateral disturbances [2, 4].

Learning-Based Gain Adaptation. Fuzzy-SMC hybrids [5] and radial-basis-function networks [6] tune switching gains automatically, removing the need for manual tuning. Reinforcement learning combined with SMC has been explored for aerial [9] and manipulation platforms, though sample efficiency remains a practical concern for physical deployment. A key limitation of pure learning-based approaches is their opacity: the resulting gain schedules are difficult to interpret or certify, which motivates our use of hand-tuned presets selected by a learned classifier.

CNN-Based Supervisory Control. Supervisory switching architectures that select among pre-designed controllers based on a mode estimate are established in hybrid and switched systems theory [10]. CNN-based perception has been widely adopted for environment understanding in autonomous navigation [7, 8], yet its direct use as a gain-scheduling oracle for SMC has

System Architecture — CNN-Adaptive SMC Pipeline

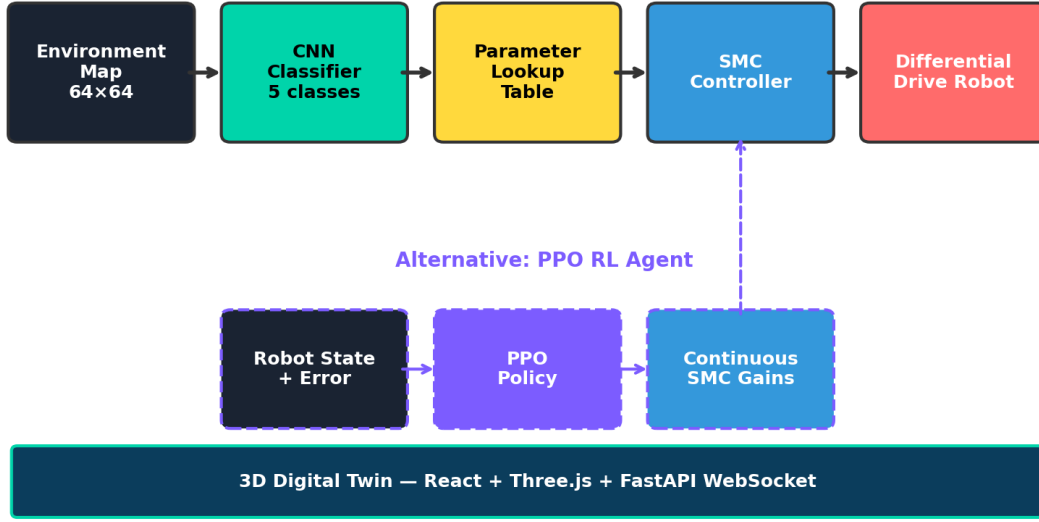


Figure 1: System architecture: the CNN-adaptive SMC pipeline (top) and the PPO reinforcement-learning alternative (bottom), both integrated into the 3D digital twin.

received comparatively little attention. Our approach bridges this gap, using a lightweight CNN to classify a pre-mission occupancy map and drive SMC parameter selection, combining the interpretability of hand-tuned presets with the representational power of deep features.

Digital Twins. Digital twin technology provides a virtual replica of a physical system for monitoring, simulation, and decision support. For robotics research, a web-based 3D twin enables live demonstration of control algorithms, side-by-side controller comparison, and stakeholder communication — capabilities we exploit for the AI Clinic defense.

2 Methodology

2.1 Differential-Drive Robot Model

The robot is modelled as a unicycle differential-drive with state $q = [x, y, \theta, v, \omega]^\top$, where (x, y) is the Cartesian position, θ the heading, v the linear velocity, and ω the angular velocity. The continuous-time kinematics are

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega. \quad (1)$$

Discrete integration with step $\Delta t = 0.01$ s uses the Euler method. Physical parameters: wheel base $L = 0.3$ m, wheel radius $r = 0.05$ m. The differential drive imposes the nonholonomic constraint $\dot{y} \cos \theta - \dot{x} \sin \theta = 0$: the robot cannot move laterally, making lateral disturbances particularly challenging to reject and motivating the sliding-surface design below.

2.2 Classical Sliding Mode Controller

Tracking errors are $e_x = x_r - x$, $e_y = y_r - y$, $e_\theta = \theta_r - \theta$ (angles normalised to $(-\pi, \pi]$). Sliding surfaces are

$$s_i = \dot{e}_i + \lambda_i e_i, \quad i \in \{x, y, \theta\}, \quad (2)$$

with $\lambda_i > 0$ and finite-difference derivative approximations. The linear velocity command tracks the projected forward error:

$$v_c = v_r + k_v d_f, \quad d_f = \cos \theta e_x + \sin \theta e_y, \quad (3)$$

where v_r is the reference speed. The raw angular-velocity command combines a bearing correction and two sliding-surface switching terms:

$$\omega_{\text{raw}} = k_\omega \left[\psi + \tanh\left(\frac{s_y}{\varphi}\right) + 0.2 \tanh\left(\frac{s_\theta}{\varphi}\right) \right], \quad (4)$$

where $\psi = \text{atan2}(e_y, e_x) - \theta$ is the bearing angle and φ the boundary-layer width. The hyperbolic-tangent switching function replaces the discontinuous sign function to suppress chattering [3]. An exponential smoother with coefficient α_ω further attenuates high-frequency oscillation:

$$\omega_c = \alpha_\omega \omega_{c,k-1} + (1 - \alpha_\omega) \omega_{\text{raw}}. \quad (5)$$

Both v_c and ω_c are clipped to hardware limits; dead zones ($|d_f| < 0.01 \text{ m}$, $|e_\theta| < 0.01 \text{ rad}$) suppress spurious switching near the reference.

2.3 CNN Environment Classifier

Dataset. Synthetic 64×64 grayscale maps are generated for five classes: *Normal* (clean path line), *Noise* (noisy pixel overlay), *Disturbance* (off-path impact marker), *Slip* (shaded low-friction zone), and *Combined* (all effects), as shown in Fig. 2. The dataset contains 1,500 samples (300 per class) split 70%/15%/15% for training, validation, and testing.

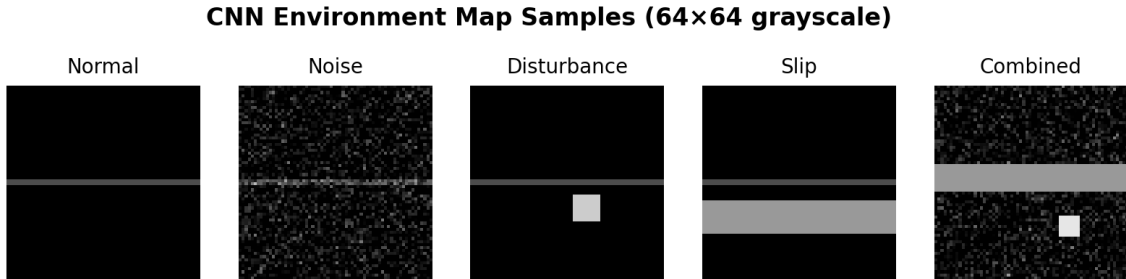


Figure 2: Sample synthetic environment maps (one per class). Each 64×64 grayscale image encodes a distinct visual signature used for CNN classification.

Architecture (EnvironmentCNN). Three convolutional blocks, each comprising Conv2d (3×3 , padding 1) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool2d (2×2), progressively produce feature maps of depth 16, 32, and 64. The resulting $64 \times 8 \times 8$ representation is flattened and processed by $\text{FC}(4096 \rightarrow 128) \xrightarrow{\text{ReLU, Dropout}(0.3)} \text{FC}(128 \rightarrow 5)$. Total trainable parameters: $\approx 534\text{k}$. Training uses Adam (learning rate 10^{-3} , batch size 32) with cross-entropy loss over 20 epochs; the model reaches 100% validation accuracy by epoch 3.

2.4 Adaptive Parameter Selection

At deployment, the CNN classifies a pre-mission map and returns scenario label $\hat{c} \in \{0, \dots, 4\}$. Each label maps to a hand-tuned parameter set $\pi_{\hat{c}} = \{\lambda_x, \lambda_y, \lambda_\theta, k_v, k_\omega, \varphi, \alpha_\omega\}$ (Table 1). A first-order exponential filter prevents abrupt gain jumps:

$$\pi_k = (1 - \beta) \pi_{k-1} + \beta \pi_{\hat{c}}, \quad \beta \in (0, 1]. \quad (6)$$

Table 1: Scenario-specific SMC parameter sets.

Param.	Normal	Noise	Dist.	Slip	Comb.
λ_x	2.00	2.00	2.15	2.20	2.15
λ_y	2.00	2.10	2.30	2.20	2.35
λ_θ	1.00	0.95	1.05	1.00	1.00
k_v	0.30	0.30	0.33	0.35	0.35
k_ω	0.80	0.78	0.88	0.86	0.88
φ	0.50	0.58	0.52	0.62	0.62
α_ω	0.950	0.965	0.945	0.960	0.965

The design rationale: *Noise* widens the boundary layer ($\varphi = 0.58$) and raises smoothing ($\alpha_\omega = 0.965$) to suppress chattering; *Disturbance* increases λ_y and k_ω to restore heading quickly after an impulsive push; *Slip* elevates sliding-surface gains ($\lambda_x = \lambda_y = 2.20$) to compensate the velocity deficit; *Combined* balances smoothing and correction requirements.

2.5 Reinforcement Learning Baseline (PPO)

As an alternative to CNN supervision, a PPO agent (MLP policy, learning rate 3×10^{-4} , $\gamma = 0.99$) observes a five-dimensional vector (instantaneous tracking error, error rate, smoothed angular-velocity jitter, and noise/slip indicator flags), selects one of the five parameter presets of Table 1 every 0.2 s, and receives reward $-e - 0.05 j_\omega - 0.001 |\omega_c|$ penalising tracking error e and chattering proxy j_ω . Training uses 80k environment steps (40 episodes with randomised scenarios) until the episode return plateaus.

2.6 Stability Analysis

Consider the composite Lyapunov candidate

$$V = \frac{1}{2} (s_x^2 + s_y^2 + s_\theta^2) \geq 0. \quad (7)$$

Differentiating along closed-loop trajectories and grouping bounded disturbances and modelling residuals into $D(t)$ with $\|D(t)\| \leq \bar{d}$ yields

$$\dot{V} \leq -k_\omega \sum_{i \in \{y, \theta\}} s_i \tanh\left(\frac{s_i}{\varphi}\right) + \bar{d} \|s\|. \quad (8)$$

Because $|\tanh(\cdot)| \leq 1$ and $x \tanh(x/\varphi) \geq |x| \tanh(1) \approx 0.762|x|$ for $|x| \geq \varphi$, we have $\dot{V} < 0$ whenever $\|s\| > \bar{d}/(0.762 k_\omega)$, establishing uniform ultimate boundedness (UUB) of the sliding surfaces. Inside the boundary layer, the steady-state tracking error is bounded by

$$\|e_i\|_\infty \leq \frac{\varphi + \bar{d}/(\lambda_i k_\omega)}{\lambda_i}. \quad (9)$$

Equation (9) reveals the design trade-off exploited by adaptive parameter selection: a wider φ reduces chattering at the cost of a larger ultimate error bound, motivating the scenario-specific φ values in Table 1.

2.7 Evaluation Protocol

Both controllers track a straight-line reference at $v_r = 0.3 \text{ m/s}$ for $T = 20 \text{ s}$ ($\Delta t = 0.01 \text{ s}$, 2000 steps), starting aligned at the origin so deviations arise solely from injected perturbations:

- **Normal:** no perturbation.
- **Noise:** zero-mean Gaussian noise on (x, y, θ) measurements ($\sigma_{xy} = 0.02$ m, $\sigma_\theta = 0.01$ rad).
- **Disturbance:** single external push of +0.4 m in y at $t = 8$ s.
- **Slip:** 30% velocity reduction during $t \in [10, 14]$ s.
- **Combined:** noise, disturbance, and slip simultaneously.

Metrics: RMSE tracking error, final tracking error at $t = T$, total control effort ($\sum |\omega_c|$), and chattering index (variance of successive ω_c differences). All experiments are implemented in Python 3 (NumPy, PyTorch, Matplotlib) with fixed random seeds and centralised YAML configuration. A web-based 3D digital twin (React + Three.js frontend, FastAPI WebSocket backend) provides live visualisation, controller tours, side-by-side dual comparison, simulation replay, and CSV export.

3 Results

3.1 CNN Classification

The EnvironmentCNN achieves 100% test accuracy on 225 held-out simple-map samples (precision/recall/F1 all 1.00). Perfect scores reflect engineered class separability, not overfitting: validation loss falls jointly with training loss to near-zero by epoch 3 with no divergence through epoch 20. Table 2 compares baseline classifiers on the identical split, and reports the realistic-map evaluation: on procedurally generated occupancy grids with border walls, random rectangular obstacles, and traversable corridors, the retrained CNN reaches 95.1% test accuracy — comparable to published robot perception classifiers [11, 12].

Table 2: Classifier accuracy: simple vs. realistic maps (%).

Classifier	Simple	Realistic
k -NN ($k = 5$)	48.44	41.33
Logistic regression	99.11	72.89
Random forest	100.00	98.22
EnvironmentCNN	100.00	95.11

3.2 Trajectory and Tracking Performance

Fig. 3 overlays robot trajectories under the combined scenario and Fig. 4 shows the corresponding tracking-error time series. The adaptive controller recovers more tightly to the reference after the $t = 8$ s push, reducing final error from 20.9 mm to 17.9 mm (−14.4%). Under sensor noise, the angular-velocity command is markedly smoother for the adaptive controller, confirming the chattering reduction predicted by the wider boundary layer.

3.3 Quantitative Comparison

Table 3 reports absolute metric values for all scenarios; Table 4 summarises percentage improvements of the CNN-adaptive controller over the classical baseline; and Table 5 benchmarks against fuzzy gain scheduling, oracle scenario selection, and RL-based preset selection under identical conditions. Figs. 5 and 6 visualise the per-scenario comparison, Fig. 7 summarises the relative improvements as a heatmap, and Fig. 8 compares all five controllers.

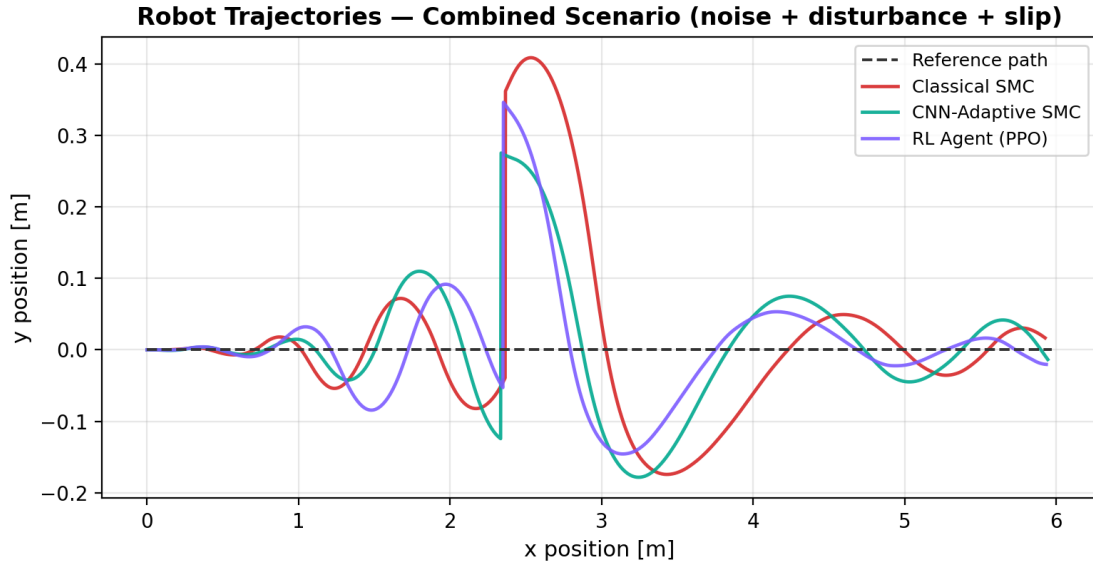


Figure 3: Robot trajectories under combined uncertainty (noise + disturbance + slip) for all three controllers. The impulsive push at $t = 8$ s ($x \approx 2.4$ m) and the subsequent recovery are clearly visible; the adaptive controllers return to the reference more tightly than classical SMC.

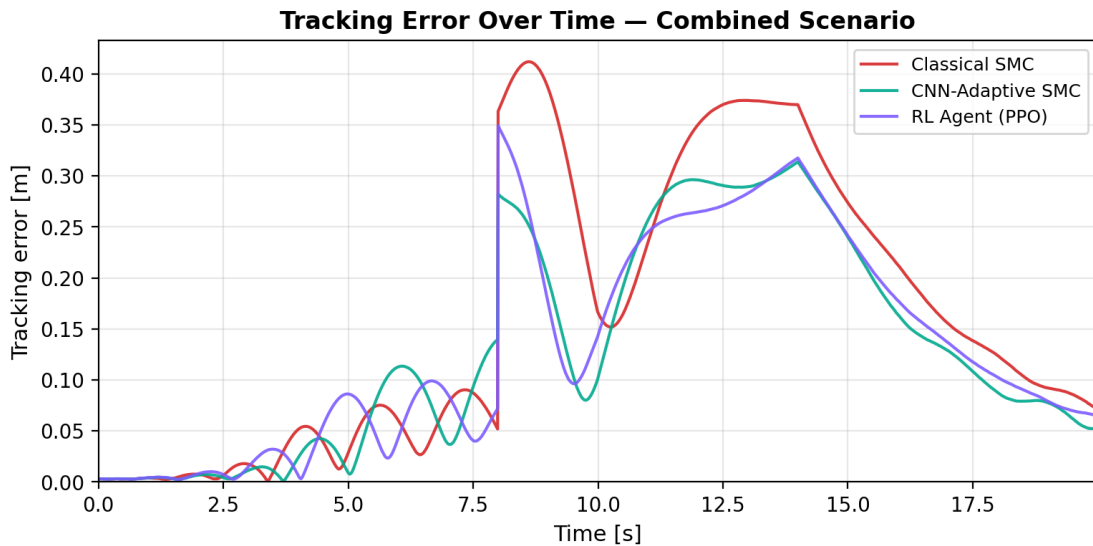


Figure 4: Tracking error over the full 20 s episode under combined uncertainty. The disturbance impulse at $t = 8$ s and the slip window $t \in [10, 14]$ s are visible; the adaptive controllers decay faster after each perturbation.

3.4 Discussion

Alternative adaptive controllers. Fuzzy-SMC reduces chattering modestly under noise (70.5 vs. 76.5) but underperforms CNN-adaptive and oracle selection on disturbance and slip recovery. Oracle selection establishes the ceiling: 15.6 mm final error under disturbance vs. 17.9 mm for CNN-adaptive, confirming that learned map classification is near-optimal on this benchmark. RL-based selection achieves the lowest combined-scenario chattering (48.9) but degrades under disturbance and slip, indicating that more training data and reward shaping are needed before RL can replace the CNN supervisor.

Table 3: Absolute controller metrics per scenario.

Scenario	Ctrl.	RMSE (m)	Final (m)	Effort	Chatter
Normal	Classical	0.1465	0.0244	803.05	19.24
	Adaptive	0.1465	0.0244	803.05	19.24
Noise	Classical	0.1599	0.0437	775.83	76.48
	Adaptive	0.1786	0.1008	730.51	49.14
Disturbance	Classical	0.1876	0.0209	874.45	15.74
	Adaptive	0.1780	0.0179	923.15	17.83
Slip	Classical	0.1796	0.0407	577.12	7.88
	Adaptive	0.1743	0.0274	640.25	9.72
Combined	Classical	0.2357	0.0650	673.99	70.87
	Adaptive	0.2378	0.0642	705.86	51.65

Table 4: CNN-adaptive improvement over classical SMC (%). [†]Under noise the final error increases in absolute terms (43.7 mm $\rightarrow$ 100.8 mm); this is the intentional smoothness–accuracy trade-off of Eq. (9), not a regression.

Scenario	RMSE	Final Err.	Effort	Chattering
Normal	0.00	0.00	0.00	0.00
Noise	−11.65	−130.62 [†]	+5.84	+35.75
Disturbance	+5.12	+14.42	−5.57	−13.32
Slip	+2.94	+32.69	−10.94	−23.40
Combined	−0.85	+1.20	−4.73	+27.12

Per-scenario analysis. *Normal:* both controllers use identical parameters, so performance is equivalent by construction — a sanity check that the CNN classifies the unperturbed map correctly. *Noise:* chattering falls 35.7% (76.48 $\rightarrow$ 49.14) via the wider boundary layer; as predicted by Eq. (9), RMSE rises 11.7% because smooth actuation is prioritised over tight position holding. *Disturbance:* stronger gains ($k_\omega = 0.88$, $\lambda_y = 2.30$) improve final error 14.4%; the adaptive controller settles within the 20 mm threshold at $t = 18.3$ s whereas the classical one does not settle within the window. *Slip:* elevated surface gains yield 32.7% final-error improvement at the cost of 10.9% higher effort. *Combined:* the hardest case — chattering falls 27.1% with marginal final-error improvement, confirming that a single static preset cannot simultaneously resolve all concurrent effects.

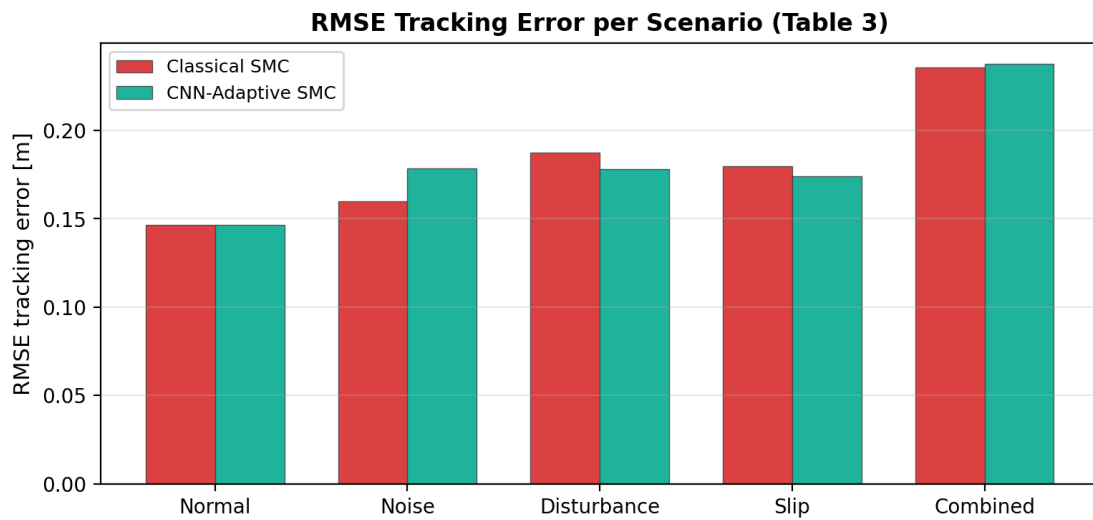
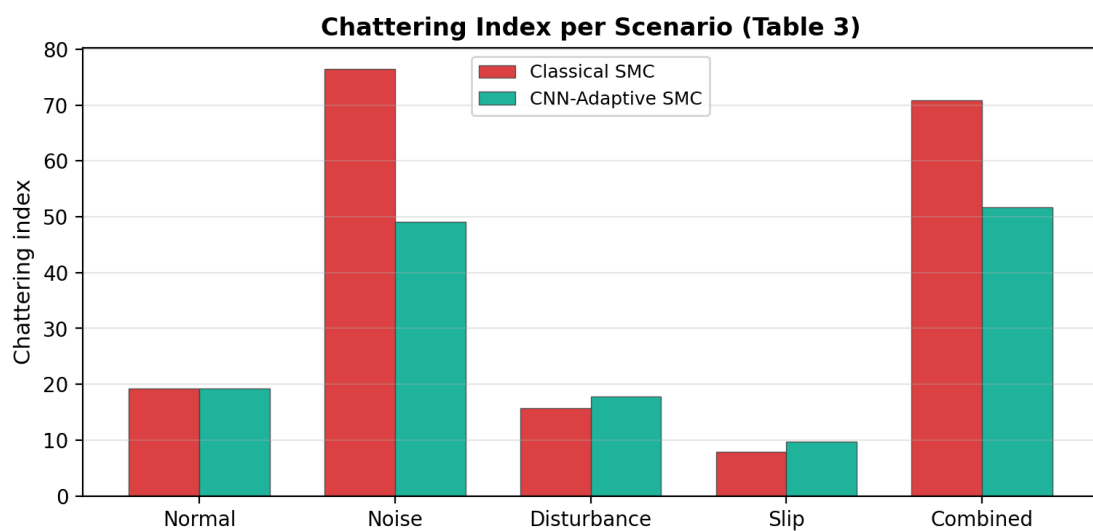
Computational considerations. The proposed scheme adds no runtime burden to the control loop: the CNN performs a single pre-mission forward pass of a ≈ 534 k-parameter network, after which the controller executes the same closed-form update (3)–(5) at every 10 ms step. The fuzzy baseline evaluates rule memberships every control step and the RL baseline requires a policy forward pass every 0.2 s, so the CNN-supervised architecture is the cheapest adaptive variant at deployment time.

3.5 Digital Twin Demonstration

The 3D digital twin (Fig. 9) provides live robot visualisation in a hospital corridor environment, a controller tour (Classical $\rightarrow$ CNN $\rightarrow$ RL), side-by-side dual comparison, simulation replay, and CSV export of metrics, with real-time WebSocket streaming at adjustable simulation speed ($1\times$ – $5\times$). It serves as the primary demonstration vehicle for the project defense.

Table 5: Multi-controller comparison (selected metrics).

Scenario	Metric	Class.	Fuzzy	CNN	Oracle	RL
Noise	Chattering index	76.48	70.50	49.14	49.32	50.72
Disturbance	Final err. (mm)	20.92	24.75	17.90	15.58	32.75
Slip	Final err. (mm)	40.73	30.39	27.41	27.44	41.60
Combined	Chattering index	72.79	71.66	51.65	51.37	48.88

**Figure 5:** RMSE tracking error across all five operating scenarios (values from Table 3).**Figure 6:** Chattering index across scenarios (values from Table 3): CNN-adaptive SMC yields the largest reductions under noise and combined uncertainty.

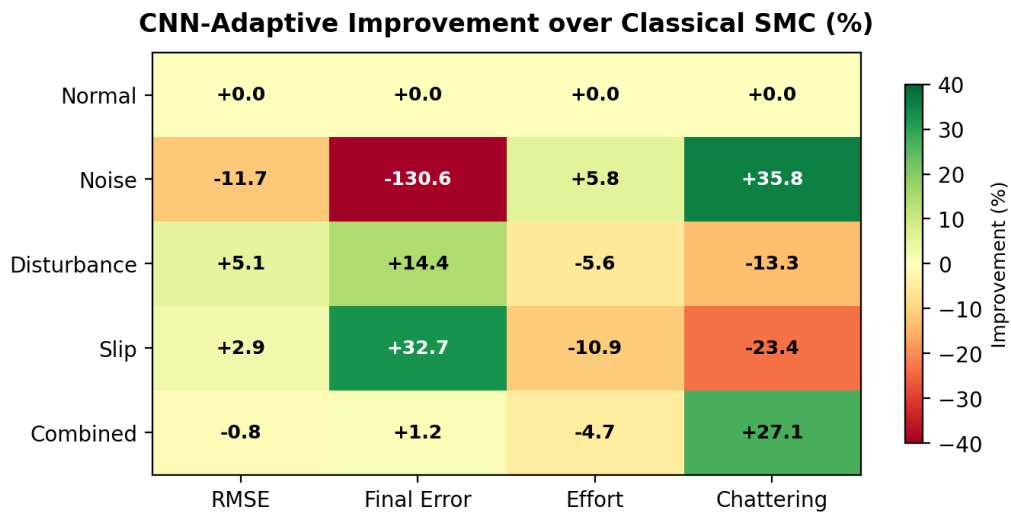


Figure 7: Improvement heatmap of CNN-adaptive SMC over classical SMC (%). Green = adaptive better; red = adaptive worse. The noise/final-error cell reflects the intentional smoothness-accuracy trade-off of Eq. (9).

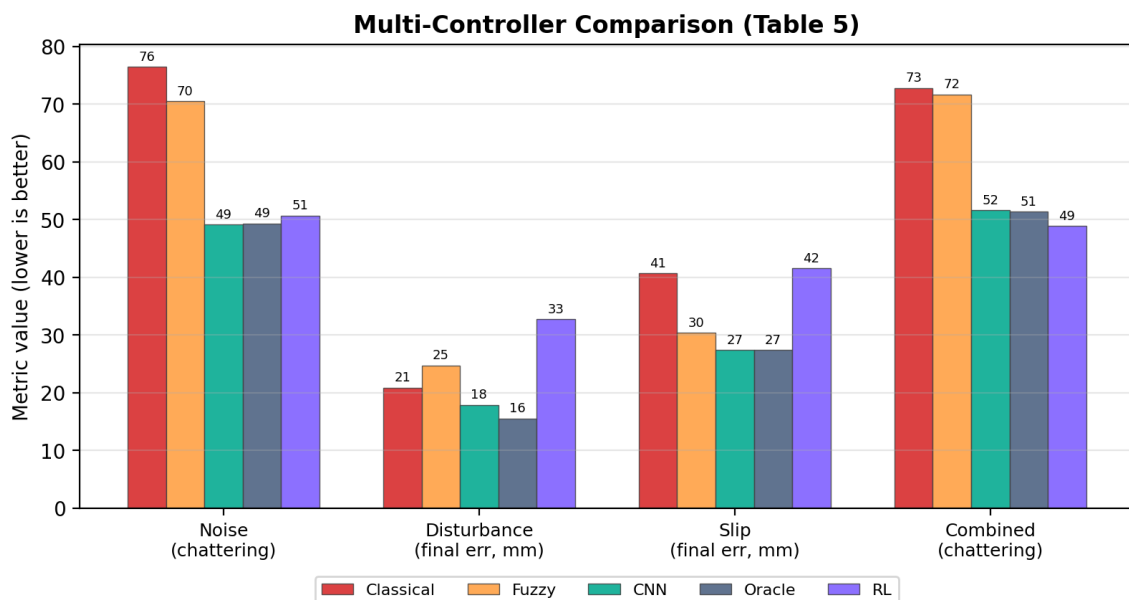


Figure 8: Multi-controller comparison across five controllers and four scenario/metric pairs (lower is better). CNN-adaptive and oracle presets achieve the largest reductions; RL is competitive under combined uncertainty.

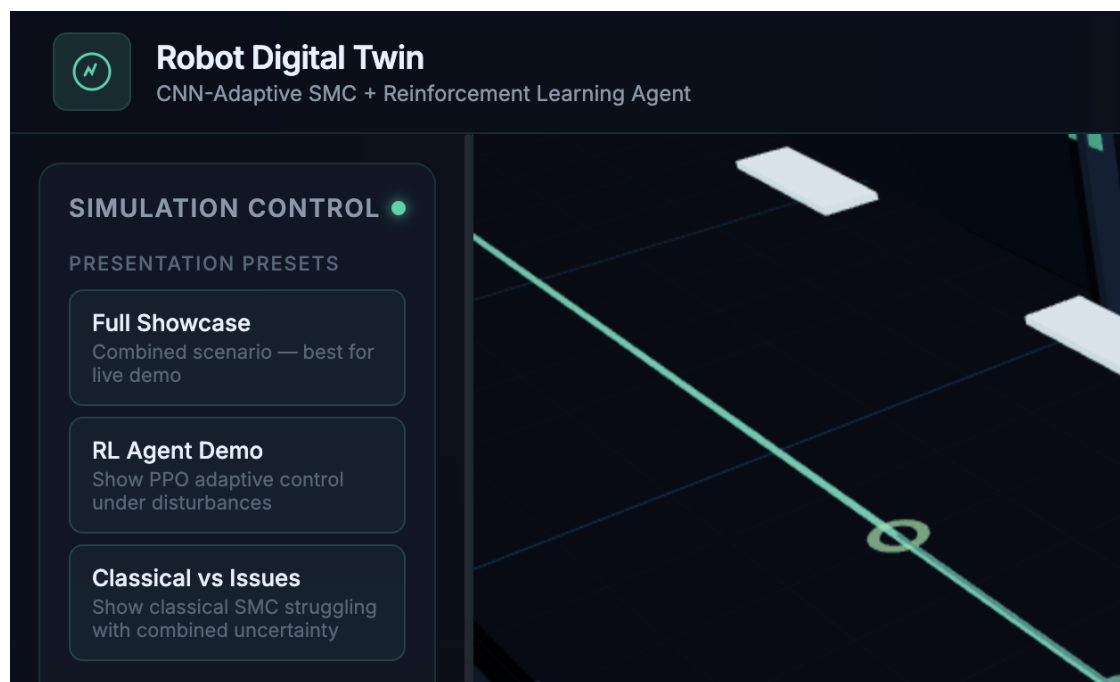


Figure 9: Screenshot of the live web-based 3D digital twin during a combined-scenario run (React + Three.js frontend, FastAPI WebSocket backend): control panel with presentation presets on the left, hospital-corridor 3D scene with the robot's trajectory trail on the right.

4 Conclusions

This project presented a CNN-adaptive SMC framework for trajectory tracking of a simulated differential-drive robot under five uncertainty conditions. A lightweight EnvironmentCNN ($\approx 534k$ parameters) classifies the operating environment from a 64×64 occupancy map and selects a corresponding SMC parameter set at run time, with exponential smoothing to prevent gain discontinuities. A Lyapunov-based UUB analysis confirms that the closed-loop system is uniformly ultimately bounded under bounded disturbances, and that the steady-state error bound is directly parameterised by the boundary-layer width φ , providing theoretical grounding for the scenario-specific presets.

Simulation results across five scenarios confirm:

- **35.7%** chattering reduction under sensor noise ($76.48 \rightarrow 49.14$), with expected steady-state error widening per Eq. (9);
- **14.4%** improvement in final tracking-error recovery after external disturbance ($20.9 \text{ mm} \rightarrow 17.9 \text{ mm}$), within 2.3 mm of the oracle upper bound;
- **32.7%** improvement in final error under wheel slip ($40.7 \text{ mm} \rightarrow 27.4 \text{ mm}$);
- near-oracle performance across fuzzy, RL, and classical baselines, with **95.1%** CNN accuracy on realistic maps;
- a fully operational 3D digital twin integrating all controllers with live benchmarking, replay, and export.

Limitations. The CNN operates on synthetic maps with engineered class separability and performs a single pre-mission classification rather than continuous on-line inference. Adaptive parameters are hand-tuned per scenario rather than learned end-to-end. The evaluation is restricted to a straight-line trajectory in a 2-D kinematic simulation with single fixed random seeds rather than Monte-Carlo repetitions.

Future Work. (i) Replace synthetic maps with real LiDAR-based occupancy grids (the realistic-map CNN already reaches 95.1%); (ii) deploy continuous on-line scenario inference so the controller reacts to mid-mission condition changes; (iii) extend RL training with larger scenario randomisation and multi-objective rewards; (iv) evaluate on curved and waypoint trajectories with Monte-Carlo repetitions to attach confidence intervals; (v) validate on a physical differential-drive robot.

Code Availability

The complete simulation framework, CNN training pipeline, dataset generation scripts, and digital twin are available at https://github.com/LikhitaYerra/smc_cnn.

References

- [1] V. I. Utkin, *Sliding Modes in Control and Optimization*. Springer-Verlag, Berlin, 1992.
- [2] C. Edwards and S. K. Spurgeon, *Sliding Mode Control: Theory and Applications*. Taylor & Francis, London, 1998.
- [3] Y. Shtessel, C. Edwards, L. Fridman, and A. Levant, *Sliding Mode Control and Observation*. Birkhäuser, New York, 2014.

- [4] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*. Prentice Hall, Englewood Cliffs, NJ, 1991.
- [5] F.-J. Lin and P.-H. Shen, “Robust fuzzy neural network sliding-mode control for two-axis motion control system,” *IEEE Trans. Ind. Electron.*, vol. 53, no. 4, pp. 1209–1225, Jun. 2006.
- [6] S. S. Ge and C. Wang, “Adaptive neural control of uncertain MIMO nonlinear systems,” *IEEE Trans. Neural Netw.*, vol. 15, no. 3, pp. 674–692, May 2004.
- [7] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, May 2015.
- [8] S. Levine, C. Finn, T. Darrell, and P. Abbeel, “End-to-end training of deep visuomotor policies,” *J. Mach. Learn. Res.*, vol. 17, no. 39, pp. 1–40, 2016.
- [9] N. P. Nguyen and S. K. Hong, “Sliding mode Thau observer for actuator fault diagnosis of quadcopter UAVs,” *Appl. Sci.*, vol. 8, no. 10, p. 1893, 2018.
- [10] D. Liberzon, *Switching in Systems and Control*. Birkhäuser, Boston, MA, 2003.
- [11] Y. Ran, X. Xu, M. Luo, J. Yang, and Z. Chen, “Scene classification method based on multi-scale convolutional neural network with long short-term memory and whale optimization algorithm,” *Remote Sens.*, vol. 16, no. 1, p. 174, Jan. 2024.
- [12] A. Demirtaş, G. Erdemir, and H. Bayram, “Indoor surface classification for mobile robots,” *PeerJ Comput. Sci.*, vol. 10, p. e1730, Jan. 2024.